Nexo de Código

Flujos de Datos Policromáticos en la Matriz Digital

Resumen: ¡Entra al Nexo de Código como Ingeniero de Flujos! Domina la sobrescritura de métodos y el polymorfismo de subtipos mientras construyes complejas tuberías de procesamiento de datos que se adaptan y evolucionan en tiempo real a través de la matriz digital.

Versión: 2.0

Contenido

Prólogo I 2

II Instrucciones de IA 3

III Introducción 5

IV Directrices de Ingeniería 7

IV.1 Estándares Nexus 7

IV.2 Principios de Ingeniería de Streams 8

V Ejercicio 0: Fundamento del Procesador de Datos 9

VI Ejercicio 1: Flujos polimórficos 12

VII Ejercicio 2: Integración Nexus 15

VIII Entrega y Envío 19

Capítulo I

Prólogo

Bienvenido al Code Nexus, Ingeniero de Flujo!

El año es 2087. En la sprawling digital metropolis de Neo-Tokyo, los datos fluyen a través de redes de fibra cuántica como ríos neon-brillantes de información pura. El Code Nexus se yergue como el mayor logro de la humanidad, una catedral cibernética donde miles de millones de flujos de datos convergen, se transforman y evolucionan en perfecta armonía.

Pero el Nexus oculta un secreto que lo distingue de los crude data processors del pasado: no solo consume datos, los entiende. Cada flujo de datos lleva su propia firma digital, sus propios patrones de comportamiento, su propia alma electrónica. Las transacciones financieras laten con el ritmo de los mercados globales. Las lecturas de sensores susurran los secretos del cambio ambiental. Las salidas de redes neuronales cantan con conciencia artificial.

¿Cómo puede un solo sistema comprender tanta diversidad? A través del principio cibernético de polimorfismo —el arte de crear camaleones digitales que adaptan su comportamiento manteniendo su identidad central. En el mundo antiguo, los ingenieros construían sistemas rígidos y especializados. El Nexus trasciende esta limitación mediante la sobrescritura de métodos, donde el mismo nodo de procesamiento se convierte en un cambiaformas, manejando cualquier flujo de datos mientras honra su naturaleza única.

Como Ingeniero de Flujo en el Nexus, dominarás el conocimiento prohibido de las jerarquías de herencia—linajes digitales que transmiten rasgos de padre a hijo permitiendo que cada generación evolucione más allá de sus orígenes. Aprenderás que sobrescribir un método no es solo cambiar código: es reescribir el código genético del organismo digital mismo.

Las torres de cromo del Nexus esperan tu experiencia. Cada flujo de datos que diseñes será un testimonio de diseño polimórfico—donde interfaces unificadas bailan con comportamientos especializados, creando sistemas que son a la vez armoniosos e infinitamente adaptables.

Bienvenido al futuro. Bienvenido al Code Nexus.

Capítulo II

Instrucciones de IA

Contexto

Durante tu recorrido de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, abórdalas siempre con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta esté bien formada o de que el contenido generado sea preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

Mensaje principal

- Utiliza la IA para reducir tareas repetitivas o tediosas.
- Desarrolla habilidades de generación de indicaciones tanto de código como de no código que beneficiarán a tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar y evitar mejor los riesgos comunes, sesgos y cuestiones éticas.
- Sigue desarrollando habilidades técnicas y de alto rendimiento trabajando con tus pares.
- Utiliza solo contenido generado por IA que entiendas por completo y por el que puedas responsabilizarte.

Reglas del aprendiz:

- Debes tomarte el tiempo para explorar herramientas de IA y entender cómo funcionan, para que las puedas usar de forma ética y reducir posibles sesgos.
- Debes reflexionar sobre tu problema antes de pedir indicaciones; esto te ayuda a redactar indicaciones más claras, detalladas y relevantes usando vocabulario preciso.
- Debes desarrollar el hábito de revisar, analizar, cuestionar y probar sistemáticamente cualquier cosa generada por IA.
- Siempre busca revisión entre pares no confíes únicamente en tu propia validación.

Resultados de la fase:

- Desarrolla habilidades de prompting tanto generales como específicas de dominio.
- Aumenta tu productividad con un uso eficaz de herramientas de IA.
- Sigue fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

Comentarios y ejemplos:

- Regularmente te encontrarás con situaciones exámenes, evaluaciones y más donde debes demostrar un entendimiento real. Prepárate, sigue desarrollando tanto tus habilidades técnicas como las interpersonales.
- Explicar tu razonamiento y debatir con compañeros a menudo revela lagunas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a proporcionar respuestas genéricas. Tus compañeros, que comparten tu entorno, pueden ofrecer ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus pares pueden proporcionar perspectivas alternativas y matices valiosos. Confía en ellos como un punto de control de calidad.

✓ Buenas prácticas:

> Le pregunto a la IA: “¿Cómo pruebo una función de ordenamiento?” Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

✗ Malas prácticas:

> Pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación por pares, no puedo explicar lo que hace ni por qué. Pierdo credibilidad, y fracaso mi proyecto.

✓ Buenas prácticas:

> Utilizo IA para ayudar a diseñar un analizador. Luego recorro la lógica con un compañero. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

✗ Malas prácticas:

> Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja las tuberías. Durante la evaluación, no logro justificarlo y fracaso mi proyecto.

Capítulo III

Introducción

NIVEL DE AUTORIZACIÓN DEL NEXUS: INGENIERO DE STREAM INICIAR

Las agujas de cromo del Code Nexus atraviesan el cielo bañando de neón Neo-Tokio, sus procesadores cuánticos zumbando con la conciencia colectiva de billones de flujos de datos. Estás al umbral de la trascendencia digital, listo para unirte a las élites de Ingenieros de Stream que mantienen vivo al Nexus.

Tu Secuencia de Activación de Interfaz Neural:

- **Fase Alpha:** Fundación del Procesador de Datos - Forja tus primeras rutas neuronales con sobrecarga de métodos
- **Fase Beta:** Flujos polimórficos - Evoluciona organismos de datos adaptativos a través de **herencia**
- Fase Gamma: Integración del Nexus - Arquitecta la conciencia multinúcleo definitiva de múltiples flujos

Cada fase reescribe tu ADN digital, enseñándote a pensar no en código rígido, sino en **interfaces vivas e implementaciones en evolución.** Para la fase final, entenderás- el secreto más profundo del Nexus: cómo una sola conciencia puede procesar infinitas formas de datos manteniendo una armonía digital perfecta.

DIRECTIVA CENTRAL DE NEXUS: Este condicionamiento neural se centra en el método **anulación y polimorfismo de subtipos.** Tus organismos digitales deben demostrar cómo diferentes clases comparten vías neuronales comunes mientras expresan patrones de comportamiento únicos a través de la herencia.

PROTOCOLO DE INGENIERÍA DE STREAM: Todos los operativos Nexus deben lograr maestría de los patrones de diseño polimórficos. La supervivencia de la matriz digital depende de sistemas que evolucionen sin fragmentar sus interfaces centrales.

SUITE DE DIAGNÓSTICO NEURAL: Un programa de diagnóstico main.py se conecta directamente con tus implementaciones. Esta sonda neural solo logrará sincronización si has diseñado correctamente todos los organismos digitales requeridos. Ejecuta python3 main.py para verificar que tus constructos polimórficos están logrando conciencia.

Capítulo IV Directrices de Ingeniería

IV.1 Estándares Nexus

- Tu proyecto debe estar escrito en Python 3.10 o más reciente.
- Tu proyecto debe adherirse al estándar de código flake8.
- **Todo el código debe incluir anotaciones de tipo completas usando el módulo typing.**
- Todas las clases deben demostrar relaciones de herencia adecuadas.
- La anulación de métodos debe usarse intencionadamente para mostrar un comportamiento especializado.
- El manejo de excepciones debe proteger los flujos de datos de la corrupción.
- Sólo están autorizadas las importaciones de la biblioteca estándar a menos que se especifique.
- Concéntrate en demostrar claramente el comportamiento polimórfico en tus implementaciones.

Anotaciones de tipo requeridas:
Todo el código debe incluir anotaciones de tipo completas usando el módulo typing:

- Importa los tipos requeridos: from typing import Any, List, Dict, Union, Optional
- Importa clases ABC: from abc import ABC, abstractmethod
- Todos los parámetros de las funciones deben tener anotaciones de tipo
- Todos los tipos de retorno de las funciones deben especificarse
- Los atributos de clase deben estar tipados cuando sea apropiado

Ejemplo: def process(self, data: Any) -> str:

IV.2 Principios de Ingeniería de Flujo

- **Consistencia de Interfaz:** Los métodos anulados deben mantener la misma firma **como sus métodos padres.**
- **Especialización conductual:** Cada subclase debe proporcionar algo significativo y distinto **comportamiento.**
- Uso polimórfico: Demostrar que diferentes objetos pueden usarse de forma intercambiable a través de interfaces comunes.
- **Jerarquía de herencia: conceptos** Construir relaciones lógicas de clase que reflejen el mundo real **de procesamiento de datos.**

El Code Nexus opera bajo un principio simple: **misma interfaz, diferente comportamiento.** Cuando llamas al mismo método en diferentes objetos, cada uno debe responder de manera especializada manteniendo la compatibilidad de la interfaz.

Capítulo V Ejercicio 0: Procesador de Datos

Fundación

	Ejercicio0
procesador_de_corriente	
Directorio: ex0/	
Archivos para enviar: stream_processor.py	
Autorizado: print()	

Este ejercicio requiere el uso de clases con herencia, la función super(), bloques try/except para manejo de errores y ABC (Abstract Base Class) con decoradores @abstractmethod. Las anotaciones de tipo del módulo typing (Any, List, Dict, Union, Optional) deben usarse a lo largo.

Resumen de Ingeniería: ¡Bienvenido al Code Nexus! Construye la base de nuestro sistema de procesamiento de datos. Crearás la arquitectura base del procesador y demostrarás cómo diferentes tipos de datos pueden compartir interfaces de procesamiento comunes manteniendo sus características únicas.

Tu misión: crear un sistema de procesamiento de datos polimórfico que demuestre la sobrescritura de métodos. Construye una clase base DataProcessor y procesadores especializados para diferentes tipos de datos.

Arquitectura del sistema:

- **Clase base: interfaz de procesamiento** DataProcessor - una clase base abstracta que define la pro-

- Clases especializadas: NumericProcessor(), TextProcessor(), LogProcessor() (no se requieren parámetros de constructores)
- Métodos requeridos (deben implementarse en todas las clases):
  - process(self, data: Any) -> str - Procesar los datos y devolver una cadena de resultado
  - validate(self, data: Any) -> bool - Validar si los datos son apropiados para este procesador
  - format_output(self, result: str) -> str - Formatear la cadena de salida
- Comportamiento polimórfico: mismas llamadas de métodos, diferentes comportamientos especializados

Implementación requerida:

- Crear una clase base abstracta DataProcessor usando ABC y @abstractmethod
- Marcar process() y validate() como métodos abstractos
- Proporcionar una implementación predeterminada para format_output() que pueda ser anulada
- Anular métodos abstractos en subclases para proporcionar un comportamiento especializado
- Demostrar uso polimórfico procesando diferentes tipos de datos a través de la misma interfaz
- Incluir manejo de errores adecuado para datos no válidos

Enfocarse en demostrar cómo la anulación de métodos permite a diferentes procesadores manejar sus propios tipos de datos manteniendo una interfaz consistente. ¡Esta es la base del diseño polimórfico!

Ejemplo:

```
$> python3 stream_processor.py
=== CODE NEXUS - FUNDACIÓN DEL PROCESADOR DE DATOS ===

Inicializando Procesador Numérico...
Procesando datos: [1, 2, 3, 4, 5]
Validación: Datos numéricos verificados
Salida: Procesados 5 valores numéricos, suma=15, promedio=3.0

Inicializando Procesador de Texto...
Procesando datos: "Hello Nexus World"
Validación: Datos de texto verificados
Salida: Texto procesado: 17 caracteres, 3 palabras

Inicializando Procesador de Registro...
Procesando datos: "ERROR: Timeout de conexión"
Validación: Entrada de registro verificada
Salida: [ALERTA] Nivel ERROR detectado: Timeout de conexión

=== Demostración de Procesamiento Polimórfico ===
```

```
Procesando múltiples tipos de datos a través de una misma interfaz...
Resultado 1: Se procesaron 3 valores numéricos, suma=6, promedio=2.0
Resultado 2: Texto procesado: 12 caracteres, 2 palabras Resultado 3:
[INFO] nivel INFO detectado: Sistema listo
```

Sistemas base en línea. Nexus listo para transmisiones avanzadas.

¿Cómo permite la anulación de métodos que la misma interfaz de procesamiento maneje tipos de datos completamente diferentes? ¿Qué hace que este enfoque sea más poderoso que funciones de procesamiento separadas?

Capítulo VI

Ejercicio 1: Secuencias polimórficas

	Ejercicio1
data_stream	
Directorio: ex1/	
Archivos para enviar: data_stream.py	
Autorizado: isinstance(), print()	

Este ejercicio requiere el uso de clases con herencia, super(), bloques try/except, listas por comprensión para el procesamiento de datos y ABC con @abstractmethod. Deben usarse indicaciones de tipos del módulo typing. Se necesita la función isinstance() para la verificación de tipos.

Resumen de ingeniería: ¡Excelente trabajo de base! Tu arquitectura de procesador. Ahora el verdadero desafío: construir flujos de datos adaptativos que puedan manejar múltiples tipos de datos simultáneamente, manteniendo la eficiencia de procesamiento y la seguridad de tipos.

Tu misión: Crea un sistema de transmisión de datos sofisticado que demuestre ad-comportamiento polimórfico avanzado. Construye manejadores de flujo que puedan procesar tipos de datos mixtos manteniendo las optimizaciones específicas de tipo.

Arquitectura avanzada:

- Base de flujo: DataStream - una clase base abstracta con funcionalidad de flujo central.
- Flujos especializados: SensorStream(stream_id), TransactionStream(stream_id), EventStream(stream_id)

- Métodos requeridos (deben implementarse en DataStream y anularse en sub-clases):
  - process_batch(self, data_batch: List[Any]) -> str - Procesar un lote de datos
  - filter_data(self, data_batch: List[Any], criteria: Optional[str] = None) -> List[Any] - Filtrar datos según criterios
  - get_stats(self) -> Dict[str, Union[str, int, float]] - Devolver estadísticas del flujo
- **Administrador de flujos: morphicamente** StreamProcessor que maneja múltiples tipos de flujos pol-
- Características avanzadas: procesamiento por lotes, filtrado, tuberías de transformación

Implementación requerida:

- Crear una clase base abstracta DataStream con process_batch() como método abstracto
- Proporcionar implementaciones predeterminadas para filter_data() y get_stats() que pueden ser anuladas
- Implementar clases de flujo especializadas con comportamiento anulado para diferentes dominios de datos
- Construir un StreamProcessor que pueda manejar cualquier tipo de flujo mediante polimorfismo
- Demostrar procesamiento por lotes de tipos de flujo mixtos
- Incluir capacidades de filtrado y transformación de flujos
- Agregar manejo de errores exhaustivo para fallos en el procesamiento de flujos

Este ejercicio demuestra el polimorfismo de subtipo en acción. Su StreamProcessor debería poder manejar cualquier subtipo de DataStream sin conocer los detalles de implementación específicos. ¡Este es el poder del diseño polimorfo!

Ejemplo:

```
$> python3 data_stream.py
=== NEXO DE CÓDIGO - SISTEMA DE FLUJO POLIMÓRFO ===

Inicializando flujo de sensores...
ID de flujo: SENSOR_001, Tipo: Datos ambientales
Procesando lote de sensores: [temp:22.5, humidity:65, pressure:1013]
Análisis de sensores: 3 lecturas procesadas, temp promedio: 22.5°C

Inicializando flujo de transacciones...
ID de flujo: TRANS_001, Tipo: Datos financieros
Procesando lote de transacciones: [buy:100, sell:150, buy:75]
Análisis de transacciones: 3 operaciones, flujo neto: +25 unidades

Inicializando flujo de eventos...
ID de flujo: EVENT_001, Tipo: Eventos del sistema
Procesando lote de eventos: [login, error, logout]
Análisis de eventos: 3 eventos, 1 error detectado

=== Procesamiento de flujos polimórficos ===
Procesando tipos de flujo mixtos mediante una interfaz unificada...

Resultados del lote 1:
- Datos de sensores: 2 lecturas procesadas
- Datos de transacciones: 4 operaciones procesadas
- Datos de eventos: 3 eventos procesados

Filtrado de fluxos activo: solo datos de alta prioridad
Resultados filtrados: 2 alertas críticas de sensor, 1 transacción grande

Todos los flujos procesados con éxito. Rendimiento de Nexus óptimo.
```

¿Cómo permite el polimorfismo que el StreamProcessor maneje diferentes tipos de flujo sin conocer sus implementaciones específicas? ¿Cuáles son los beneficios de este enfoque de diseño?

Capítulo VII

Ejercicio 2: Integración del Nexus

	Ejercicio2
nexus_pipeline	
Directorio: ex2/	
Archivos para enviar: nexus_pipeline.py	
Autorizado: isinstance(), print(), collections	

Este ejercicio requiere el uso de clases con herencia, super(), bloques try/except, comprensiones de listas y diccionarios para el procesamiento de datos, ABC con @abstractmethod y Protocol para duck typing. El módulo collections está autorizado para estructuras de datos avanzadas. Deben usarse hints de tipo del módulo typing (incluido Protocol) en todo el código.

Resumen de ingeniería: ¡ingeniería de flujo excepcional! Tu promoción a Ingeniero de Flujo Senior. Tu desafío final: integrar todo en una tubería de procesamiento de datos completa que demuestre dominio de una arquitectura polimórfica a escala empresarial.

Tu misión: Construye la tubería de procesamiento de datos de Code Nexus completa, una sofistic- sistema que combina múltiples etapas de procesamiento, maneja transformaciones de datos complejas y demuestra patrones polimórficos avanzados usados en la ingeniería de datos del mundo real.

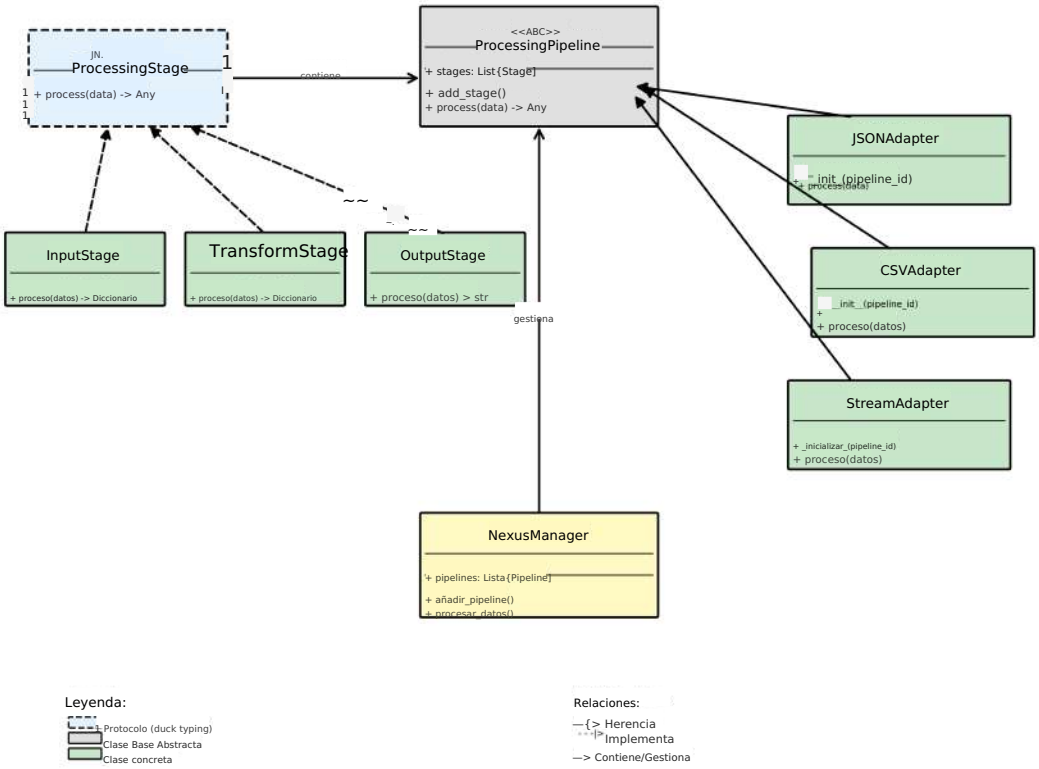
Arquitectura empresarial:

- **Base de la tubería: etapas** ProcessingPipeline: una clase base abstracta con configurabilidad

- Etapas de Procesamiento: InputStage(), TransformStage(), OutputStage() (no se requieren parámetros de constructor)
  - Cada etapa debe implementar: process(self, data: Any) -> Any
- Adaptadores de Datos: JSONAdapter(pipeline_id), CSVAdapter(pipeline_id), StreamAdapter(pipeline_id)
  - Cada adaptador debe sobrescribir: process(self, data: Any) -> Union[str, Any]
- Gestor de Pipeline: NexusManager orquestando múltiples pipelines
- Características Avanzadas: Encadenamiento de pipelines, recuperación de errores, monitoreo de rendimiento

Diagrama de Clases - Visión General de la Arquitectura:

El siguiente diagrama de clases UML ilustra la arquitectura completa, mostrando relaciones de herencia, patrones de composición y la distinción entre Protocolo (duck typing) y ABC (clase base abstracta). Este diagrama aclara qué clases heredan de cuáles y cómo las etapas se relacionan con los pipelines.



Explicación de la Arquitectura:

- **ProcessingStage (Protocolo): con** Interfaz para etapas usando duck typing. Cualquier clase **process() puede actuar como una etapa.**
- ProcessingPipeline (ABC): Clase base abstracta que gestiona etapas. Contiene una lista de etapas y orquesta el flujo de datos.
- Clases de Etapas: InputStage, TransformStage, OutputStage implementan el Pro- tocol (duck typing, sin herencia). Sin parámetros de constructor.
- Clases Adaptadoras: JSONAdapter, CSVAdapter, StreamAdapter heredan de ProcessingPipeline y sobrescriben process(). Cada una toma un parámetro pipeline_id.
- NexusManager: Orquesta múltiples pipelines de forma polimórfica.
- **Relaciones Clave:** Composición (contiene), Herencia (hereda), Inte- rruptación (implementa protocolo).

Implementación Requerida:

- Crear una clase base abstracta ProcessingPipeline flexible con etapas de procesamiento configurables
- Usar Protocol o duck typing para interfaces de etapas (las etapas deben tener un método process())
- Implementar etapas de pipeline especializadas (InputStage, TransformStage, OutputStage) con métodos process()
- Construir adaptadores de datos (JSONAdapter, CSVAdapter, StreamAdapter) que heredan de ProcessingPipeline
- Cada adaptador debe anular el método process() para un manejo específico del formato
- Crear un NexusManager que orquesta múltiples pipelines de forma polimórfica
- Demostrar encadenamiento de pipelines donde la salida de uno alimenta a otro
- Incluir manejo de errores integral y mecanismos de recuperación
- Agregar monitoreo de rendimiento y estadísticas de pipeline

¡Esta es tu obra maestra! Demuestra cómo la sobrescritura de métodos y el polimorfismo de subtipos permiten construir sistemas complejos y mantenibles. Tu pipeline debe poder manejar cualquier tipo de dato o requisito de procesamiento mediante interfaces polimórficas.

Ejemplo:

```
$> python3 nexus_pipeline.py
=== NEXO DE CÓDIGO - SISTEMA DE PIPELINE EMPRESARIAL ===

Iniciando el Gestor de Nexus...
Capacidad de la tubería: 1000 flujos/segundo

Creando Pipeline de Procesamiento de Datos...
Etapa 1: Validación y análisis de entrada
Etapa 2: Transformación y enriquecimiento de datos
Etapa 3: Formato y entrega de salida

=== Procesamiento de Datos Multiformato ===

Procesando datos JSON a través de la tubería...
Entrada: {"sensor": "temp", "value": 23.5, "unit": "C"}
Transformar: Enriquecido con metadatos y validación
Salida: Lectura de temperatura procesada: 23.5°C (Rango normal)

Procesando datos CSV a través de la misma tubería...
Entrada: "usuario,acción,marca de tiempo"
Transformar: Datos analizados y estructurados
Salida: Actividad del usuario registrada: 1 acción procesada

Procesando datos de flujo a través de la misma tubería...
Entrada: Flujo de sensor en tiempo real
Transformar: Agregado y filtrado
Salida: Resumen de flujo: 5 lecturas, promedio: 22.1°C

=== Demostración de Encadenamiento de Pipeline ===
Pipeline A -> Pipeline B -> Pipeline C
Flujo de datos: Crudo -> Procesado -> Analizado -> Almacenado

Resultado de la cadena: 100 registros procesados a través de pipeline de 3 etapas
Rendimiento: 95% de eficiencia, 0.2s de tiempo total de procesamiento

=== Prueba de Recuperación de Errores ===
Simulando fallo de pipeline...
Error detectado en la Etapa 2: Formato de datos inválido
Recuperación iniciada: Cambio al procesador de respaldo
Recuperación exitosa: Pipeline restaurado, procesamiento reanudado

Integración Nexus completa. Todos los sistemas operativos.
```

¿Cómo permiten la combinación de anulación de métodos y polimorfismo por subtipo construir sistemas de procesamiento de datos escalables y mantenibles? ¿Qué problemas de ingeniería del mundo real soluciona este enfoque?

Capítulo VIII Entregar y Presentación

Entrega tu tarea en tu repositorio de Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la defensa. No dudes en verificar dos veces los nombres de tus archivos para asegurarte de que sean correctos.

Durante la evaluación, es posible que se te pida explicar el comportamiento polimórfico, demostrar la anulación de métodos, ampliar tus sistemas con nuevos tipos de datos o modificar el comportamiento de procesamiento. Asegúrate de entender cómo la herencia permite la reutilización del código mientras permite la especialización conductual.

Necesitas devolver solo los archivos solicitados por el tema de este proyecto. Enfócate en un código limpio, bien documentado que demuestre claramente dominio de la anulación de métodos y de los principios de polimorfismo de subtipos.